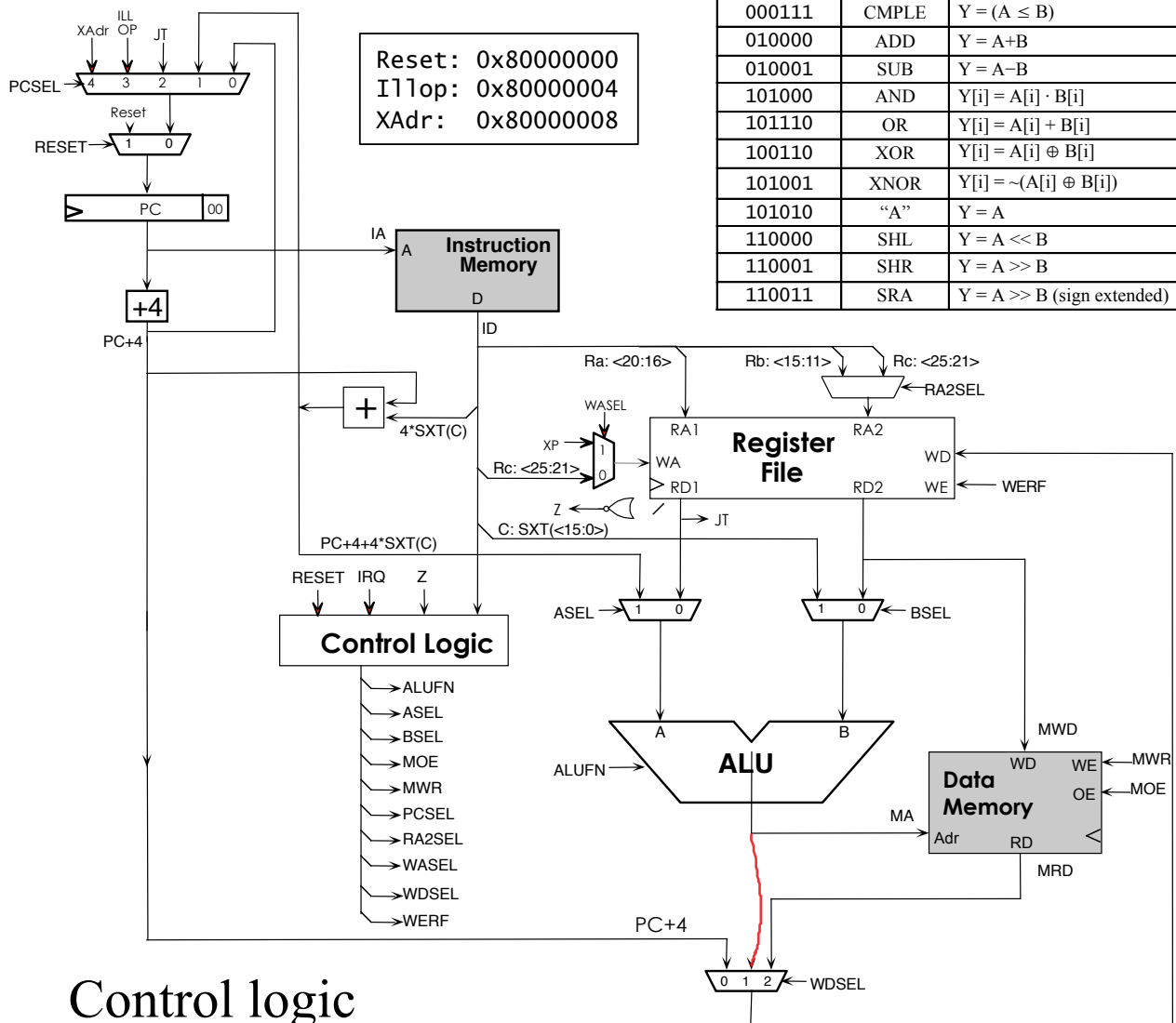


# Computation Structures

## Beta Implementation Worksheet

### Unpipelined Beta



### Control logic

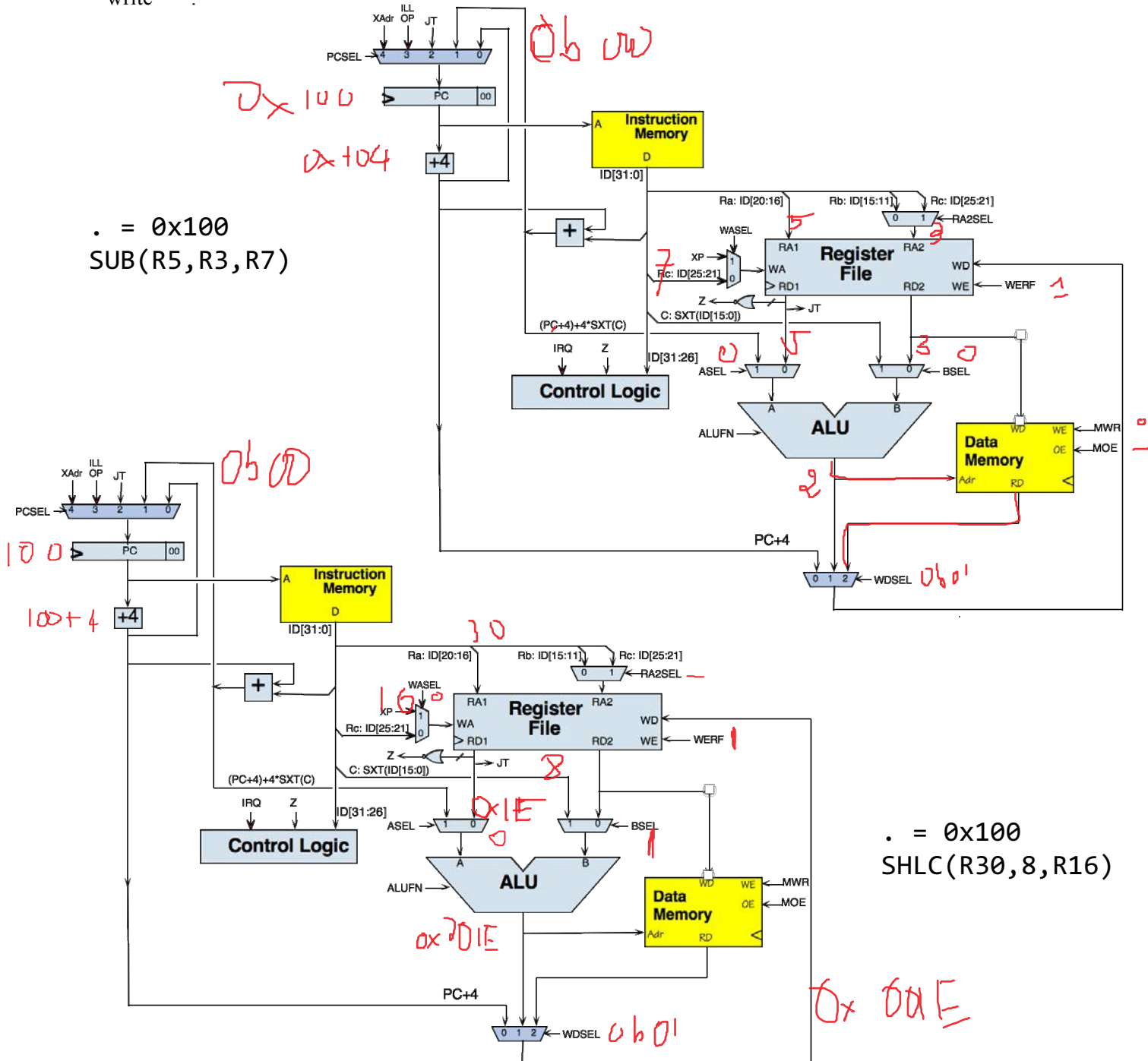
|            | RESET | IRQ | OP    | OPC   | LD  | LDR | ST  | JMP | BEQ       | BNE       | ILLOP |
|------------|-------|-----|-------|-------|-----|-----|-----|-----|-----------|-----------|-------|
| ALUFN[5:0] | --    | --  | F(op) | F(op) | "+" | "A" | "+" | --  | --        | --        | --    |
| ASEL       | --    | --  | 0     | 0     | 0   | 1   | 0   | --  | --        | --        | --    |
| BSEL       | --    | --  | 0     | 1     | 1   | --  | 1   | --  | --        | --        | --    |
| MOE        | --    | --  | --    | --    | 1   | 1   | 0   | --  | --        | --        | --    |
| MWR        | 0     | 0   | 0     | 0     | 0   | 0   | 1   | 0   | 0         | 0         | 0     |
| PCSEL[2:0] | --    | 4   | 0     | 0     | 0   | 0   | 0   | 2   | Z ? 1 : 0 | Z ? 0 : 1 | 3     |
| RA2SEL     | --    | --  | 0     | --    | --  | --  | 1   | --  | --        | --        | --    |
| WASEL      | --    | 1   | 0     | 0     | 0   | 0   | --  | 0   | 0         | 0         | 1     |
| WDSEL[1:0] | --    | 0   | 1     | 1     | 2   | 2   | --  | 0   | 0         | 0         | 0     |
| WERF       | --    | 1   | 1     | 1     | 1   | 1   | 0   | 1   | 1         | 1         | 1     |

### Problem 1.

For this problem assume that each register has been initialized to the value 0x0000??00 where “??” is the register number as a two-digit hex number. So R0 is initialized to 0x00000000, R1 to 0x00000100, ..., and R30 to 0x00001E00. R31 of course always reads as 0.

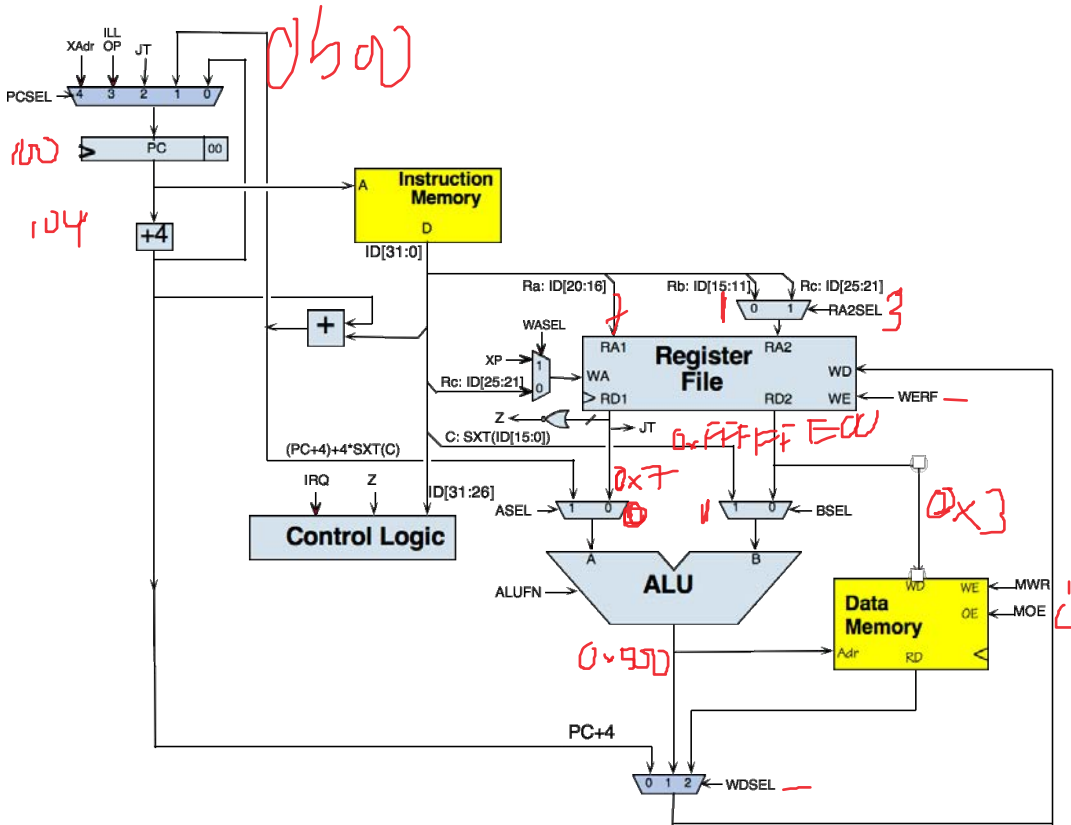
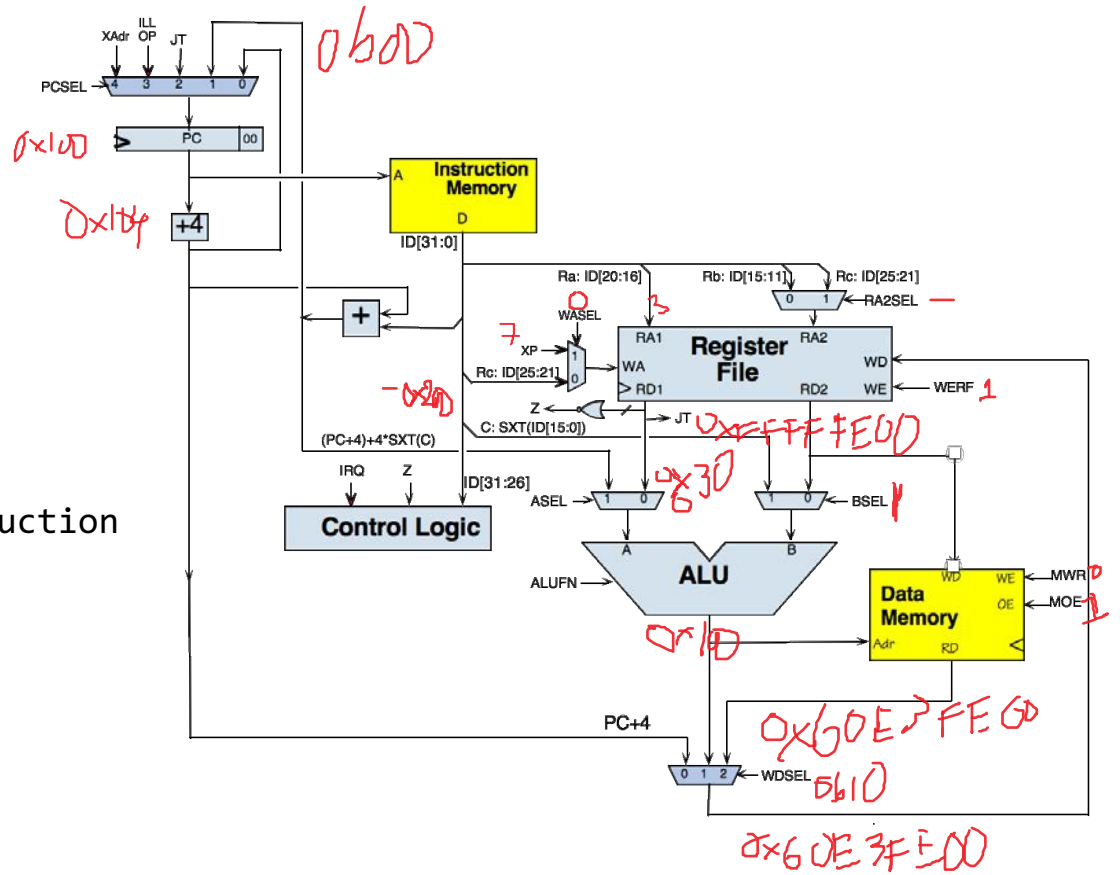
For each instruction below, please indicate the values that will be found in the unpipelined Beta datapath just before the end of the clock cycle in which the instruction is executed. If the value doesn't matter since it's not used during the execution of the instruction or can't be determined, write “—”.

. = 0x100  
SUB(R5,R3,R7)

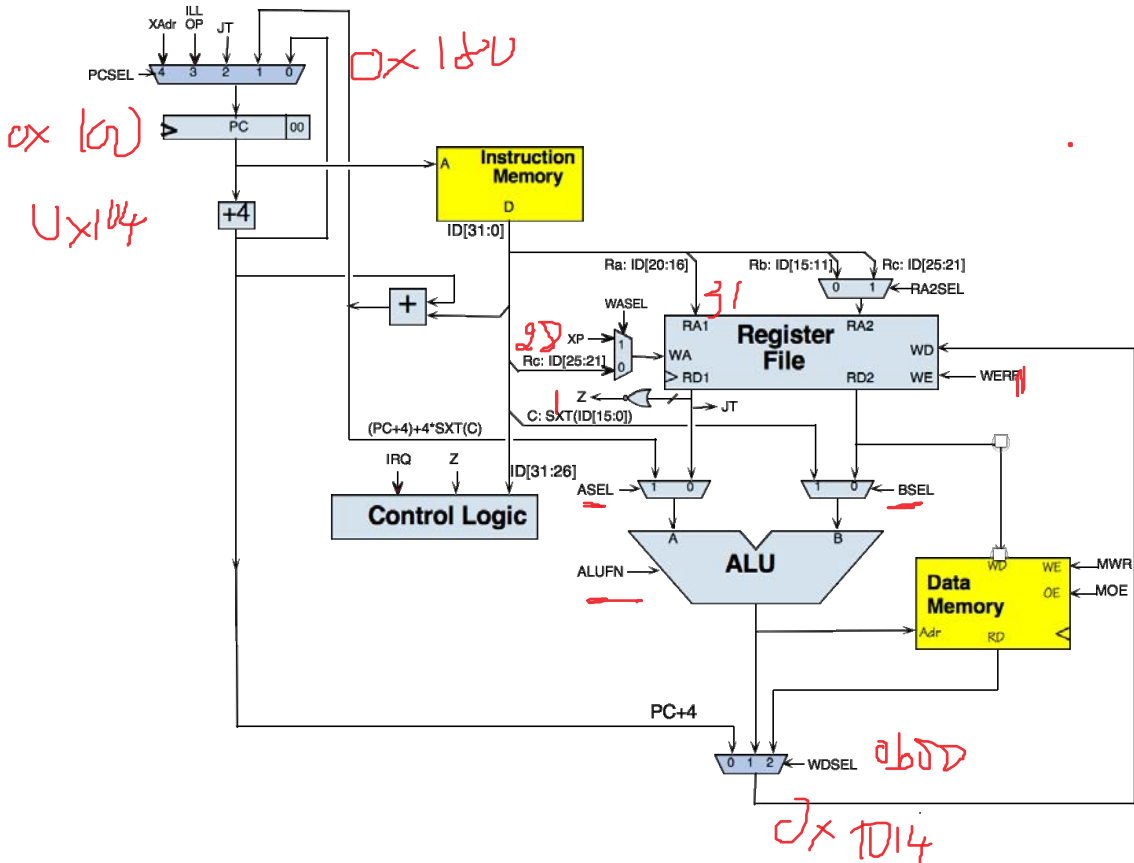
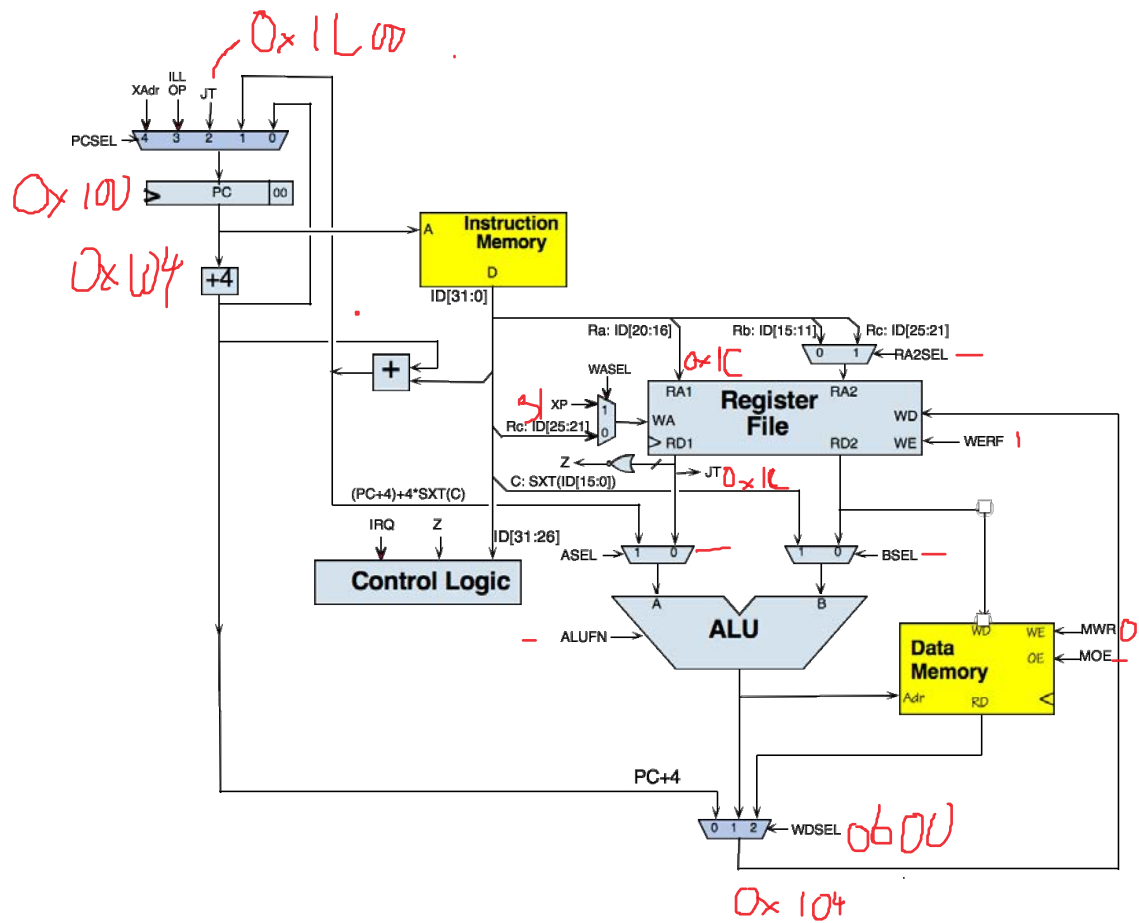


. = 0x100  
LD(R3, -0x200, R7)

// hex for instruction  
0x60E3FE00



. = 0x100  
ST(R3, -0x200, R7)



```
. = 0x100
BEQ(R31, .+0x80, LP)
```

$$\begin{aligned} 0 \times 180 &= 0 \times 104 + \text{Lit} \cdot 4 \\ \Rightarrow \text{Lit} &= \frac{0 \times 180 - 0 \times 104}{4} \\ &= 0 \times 1F \end{aligned}$$

## Problem 2.

Consider adding the following instructions to the Beta instruction set, for implementation on the Beta hardware shown in lecture (see diagram included in the reference material at the end of this quiz). You're allowed to change how the control signals are generated but no modifications to the datapath are permitted.

For each instruction either fill in the appropriate values for the control signals in the table below or **put a line through the whole row if the instruction cannot be implemented** using the existing Beta datapath. Use “—” to indicate a “don't care” value for a control signal. The values can be a function of Z (which is 1 when Reg[Ra] is zero).

**LDX( Ra, Rb, Rc )** // Load indexed

$EA \leftarrow \text{Reg}[Ra] + \text{Reg}[Rb]$   
 $\text{Reg}[Rc] \leftarrow \text{Mem}[EA]$   
 $PC \leftarrow PC + 4$

**STX( Ra, Rb, Rc )** // Store indexed

$EA \leftarrow \text{Reg}[Ra] + \text{Reg}[Rb]$   
 $\text{Mem}[EA] \leftarrow \text{Reg}[Rc]$   
 $PC \leftarrow PC + 4$

*need 3 regs*

**MVZC(Ra, literal, Rc)** // Move constant if zero

If  $\text{Reg}[Ra] == 0$  then  $\text{Reg}[Rc] \leftarrow \text{SXT}(\text{literal})$   
 $PC \leftarrow PC + 4$

**SOB(Ra, literal, Rc)** // Subtract one and branch

$PC \leftarrow PC + 4$   
 $EA \leftarrow PC + 4 * \text{SEXT}(\text{literal})$   
 $\text{tmp} \leftarrow \text{Reg}[Ra]$   
 $\text{Reg}[Rc] \leftarrow \text{Reg}[Ra] - 1$   
 if  $\text{tmp} \neq 0$  then  $PC \leftarrow EA$

*X can't have 2 literal at once time*

**ARA(Ra, literal, Rc)** // Add Relative Address

$\text{Reg}[Rc] \leftarrow \text{Reg}[Rc] + PC + 4 + 4 * \text{SEXT}(\text{literal})$   
 $PC \leftarrow PC + 4$

(FILL IN TABLE BELOW)

| Instr | ALUFN | WERF | BSEL | WDSEL | MOE | MWR | RA2SEL | PCSEL | ASEL | WASEL |
|-------|-------|------|------|-------|-----|-----|--------|-------|------|-------|
| LDX   | "+"   | 1    | 0    | 0b10  | 1   | 0   | 0      | 0     | 0    | 0     |
| STX   | <hr/> |      |      |       |     |     |        |       |      |       |
| MVZC  | "+"   | Z    | 1    | 1     | 0   | 0   | -      | 0     | 0    | 0     |
| SOB   | <hr/> |      |      |       |     |     |        |       |      |       |
| ARA   | "+"   | 1    | 0    | 1     | -   | 0   | 1      | 0     | 1    | 0     |

### Problem 3.

Ben Bitdiddle is proposing the short assembly language program shown to the right as a manufacturing test to ensure the correct operation of the Control ROM. He is assuming – and you may too – that the Beta datapath components (e.g., Memories, ALU, muxes, register file, adders) are working correctly and that any errors in execution are due to faulty signals from the Control ROM. Ben's plan is to run the program then look at the value in the memory location labeled ANS. If the value is 0x6004, the test passes, otherwise the Beta being tested is declared faulty and discarded.

For each of the following faults, indicate the value that the faulty Beta will store into ANS.

```

.=0
0 Test: LD(R31,X,R0) 0x6003
4      ADDC(R0,1,R1) 0x6004
8      BNE(R1,L1,R31)
12     ADDC(R1,1,R1) 0x6005
16 L1: ST(R1,ANS,R31)
20     HALT()
24 X:  .LONG(0x6003)
28 ANS: .LONG(0)

```

(A) RA2SEL is stuck at the value 0. *PC chosen during ST.  $R_1 + \text{Ans} = \text{Ans}$*   
*inst[15:11] = 0 = Reg 0*  
**Value stored in ANS by faulty Beta:** 0x6003

(B) WDSEL[1:0] is stuck at the binary value 00. *= 0b00  $\Rightarrow$  always chosen value  $R_0 + 4$*   
**Value stored in ANS by faulty Beta:** 0x8

(C) PCSEL[2:0] is stuck at the binary value 000. *BNE can't branch*  
*so second add started*  
**Value stored in ANS by faulty Beta:** 0x6005

### Problem 4. Beta Implementation

Consider the assembly language program shown to the right. Assume that all register values are initialized to 0, execution starts at PC=0 and halts when HALT() is executed.

This program is run on 4 different broken Betas, where each Beta has a specified control signal stuck at the specified value, i.e., the control signal value is fixed and is not affected by the value produced by the Beta's CTL module. For each broken Beta, please give the value in registers R1, R2, R3, and the location X: after the programs halts. **Assume that any don't care control signal values are 0.**

```

.= 0
0 LD(R31,X,R1) -42
4 CMPLTC(R1,0,R2)  $R_2 = 1$ 
8 BF(R2,end,R3)  $R_3 = 0xC$ 
12 SUB(R31,R1,R1)
16 ST(R1,X,R31)
20 END: HALT()
24 X: LONG(-42)

```

| Broken control signal | Final value in |     |     |             |
|-----------------------|----------------|-----|-----|-------------|
|                       | R1             | R2  | R3  | Location X: |
| RA2SEL stuck at 0     | 42             | 1   | 0xC | 0           |
| WDSEL stuck at 0b00   | 0x10           | 0x8 | 0xC | 0x10        |
| WASEL stuck at 1      | 0              | 0   | 0   | -42         |
| WERF stuck at 1       | 0x14           | 1   | 0xC | 42          |

*of latches write to X*

*$x_4 + 12 R_0$   
 $0x10 \text{ (X/Y) write } R_2 + 4$*

*Because of  $R_1 = 1$  and  $WDSEL = 0 \rightarrow R_1 = PC + 4$*

### Problem 5.

In this problem, you will consider a number of plausible hardware faults in an otherwise working Beta processor; you may want to consult the diagram and documentation on the backs of pages of this quiz. Each of the faults involves changing a particular output of the control logic to some new (incorrect) constant value. In each case, you are to evaluate the impact of the fault on each of the following Beta instructions:

```

I1:      ST(R0, 0x100, R1)
I2:      JMP(LP, R31)
I3:      BEQ(R31, .+4, R0)
I4:      SUB(R1, R0, R0)

```

*Handwritten notes: x(0) above I1, (0) next to I2, (0) next to I3, (0) next to I4.*

For each of the following faults, identify which (if any) of the above instructions will fail to work properly – that is, if the fault might effect the processor state (register and PC values) after the execution of the instruction. Be careful: some of these are tricky!

(A) ALUFN stuck at code for “-” (32-bit SUBTRACT)

Which instruction(s) fail? Circle all applicable, or NONE: **I1** I2 I3 I4 NONE

(B) RA2SEL stuck at 1 *not 1k: Rb = Rc*

Which instruction(s) fail? Circle all applicable, or NONE: I1 I2 I3 I4 **NONE**

(C) WERF stuck at 0

Which instruction(s) fail? Circle all applicable, or NONE: I1 I2 **I3** **I4** NONE

(D) BSEL stuck at 0

Which instruction(s) fail? Circle all applicable, or NONE: **I1** I2 I3 I4 NONE

### Problem 6.

(A) The Beta executes the assembly program below starting at location 0 and stopping when it reaches the HALT() instruction. Please give the values in the indicated registers after the Beta stops. Write the values in hex or write “CAN’T TELL” if the values cannot be determined.

```

. = 0
LD(r31, X, r0)
CMPL(r0, r31, r1)
BNE(r1, L1, r2)
ADDC(r31, 1, r0)
L1: HALT()

X: LONG(0x87654321)

```

Value left in R0 or “CAN’T TELL”: 0x **87654321**

*Handwritten note: r1 = 1*

Value left in R1 or “CAN’T TELL”: 0x **1**

Value left in R2 or “CAN’T TELL”: 0x **C**

- (B) Redo part (A) but this time assume that all the control signals going to the datapath from the control logic are stuck at logic 0, *except for WERF* which operates as expected. Note that when ALUFN[4:0] = 0b00000, the ALU computes A+B.

$PCSEL = 0 \rightarrow \text{Can't branch}$   
 $\cdot = 0$   
 LD(r31, X, r0) Value left in R0 or "CAN'T TELL": 0x 10  
 CMPLE(r0, r31, r1)  
 BNE(r1, L1, r2) Value left in R1 or "CAN'T TELL": 0x 8  
 ADDC(r31, 1, r0)  
 L1: HALT()  
 X: LONG(0x87654321) Value left in R2 or "CAN'T TELL": 0x 1C  
 $\cancel{PCSEL} = 0 \rightarrow \text{always while } PC + 4$

- (C) Bettah Beta Inc. (you can tell they're based in Boston!) is proposing a new Beta instruction TCLR that sets Rc to the current value of a memory location whose address is in Ra and writes a zero to that location, all in a single cycle. They are assuming that main memory works as it does in JSim: its read ports are combinational and the write port takes a CLK signal and performs the write at the end of the current cycle – *so the same memory location can be read and written in the same clock cycle.*

Here's their draft entry for the Beta reference manual:

|            |                                                                                                                                         |    |    |      |        |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------|----|----|------|--------|
| Usage:     | TCLR(Ra,Rc)                                                                                                                             |    |    |      |        |
| Opcode:    | 011010                                                                                                                                  | Rc | Ra | 1111 | unused |
| Operation: | $PC \leftarrow PC + 4$<br>$EA \leftarrow \text{Reg}[Ra]$<br>$\text{Reg}[Rc] \leftarrow \text{Mem}[EA]$<br>$\text{Mem}[EA] \leftarrow 0$ |    |    |      |        |

The contents of register Rc are set to the contents of the memory location whose address is in Ra. Then, at the end of the cycle, that memory location is set to 0.

Please fill in the appropriate values for the control signals that will cause the datapath to implement the correct operations OR briefly explain why TCLR cannot be implemented with the existing Beta datapath in a single cycle.

Fill in table:

| Instr | ALUFN        | WERF     | BSEL     | WDSEL    | MOE      | MWR      | RA2SEL   | PCSEL    | ASEL     | WASEL    |
|-------|--------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| TCLR  | <u>00000</u> | <u>1</u> | <u>0</u> | <u>1</u> | <u>1</u> | <u>1</u> | <u>0</u> | <u>0</u> | <u>0</u> | <u>0</u> |

$ASEL = 0, ALUFN = 0 \rightarrow MA \text{ in } \text{Reg}[Ra]$   
 $\cancel{WDSEL} = 1, \cancel{WERF} = 1, \cancel{WASEL} = 0 \rightarrow Rc = \text{Mem}[EA]$   
 $\cancel{RA2SEL} = 0, \cancel{MWR} = 1 \rightarrow \text{Reg}[Rc] = R31 = 0 \text{ in } EA$



MIT OpenCourseWare  
<https://ocw.mit.edu/>

6.004 Computation Structures  
Spring 2017

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>.